

Project: Semantic Similarity-Based Plagiarism Checker for Academic Content

Project Objectives

- Develop an AI-powered plagiarism detection system capable of identifying semantic and paraphrased plagiarism by analyzing the contextual meaning of academic text rather than relying solely on exact keyword matching.
- Implement a semantic similarity-based plagiarism checker using Sentence-BERT embeddings and cosine similarity to compare academic documents and generate an interpretable plagiarism report with similarity scores and risk assessment.

MVP Scope (Phase 1)

- Teacher-side application only.
- Compare two documents at a time.
- Supported file formats: PDF and DOCX.
- Detect semantic similarity and paraphrased plagiarism.
- Generate similarity score, risk level, and review recommendations.
- No external LLM dependency in MVP.

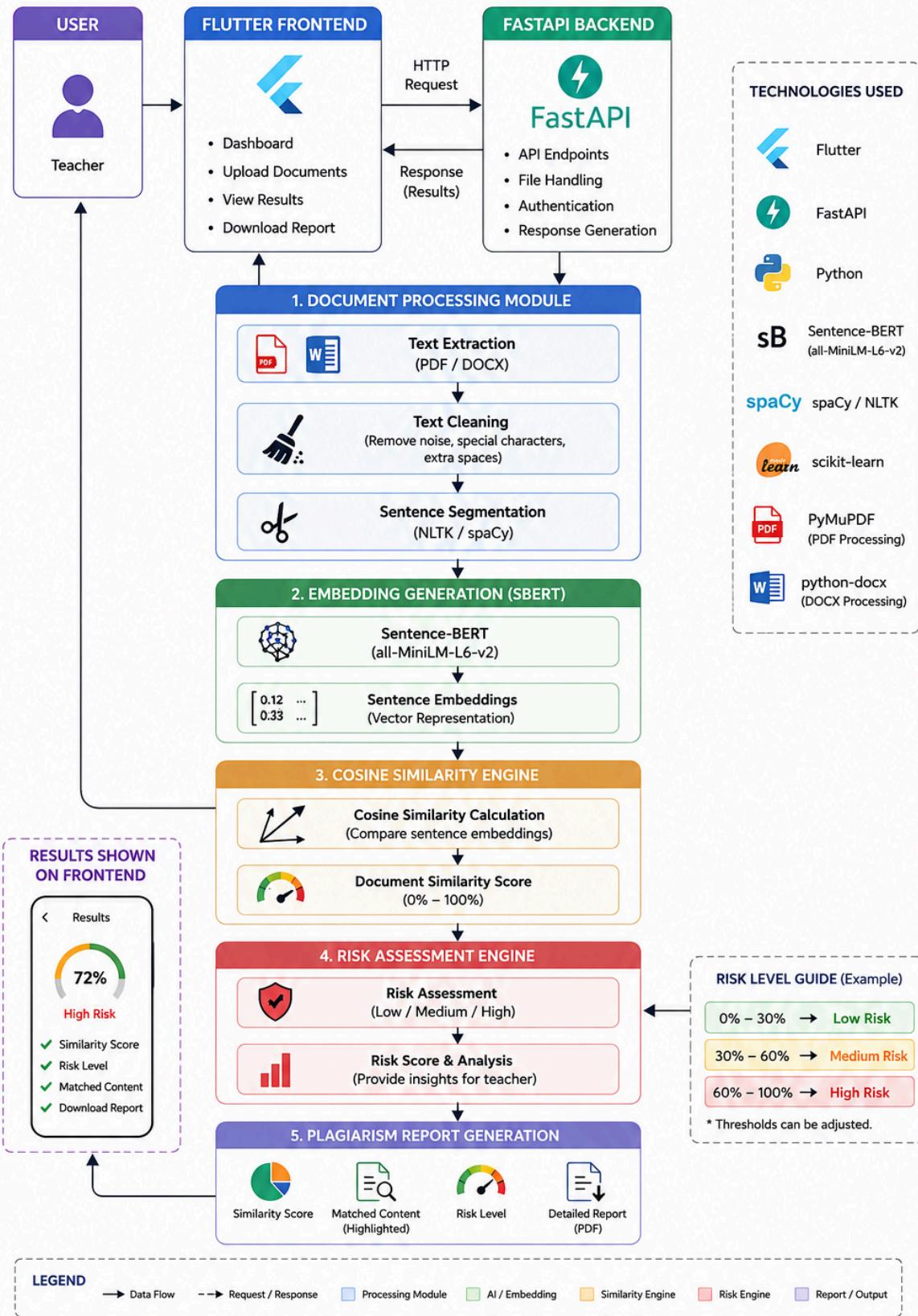
System Architecture

The following list outlines the end-to-end data flow and architectural components of the plagiarism detection system.

1. The Teacher initiates a request through the Flutter Frontend.
2. The FastAPI Backend receives the HTTP request and triggers the Document Processing Module.
3. Text is extracted from documents using PyMuPDF or python-docx.
4. Extracted text is cleaned to normalize whitespace and remove PDF artifacts.
5. Sentences are segmented using NLTK or spaCy.
6. Sentence-BERT (SBERT) generates semantic embeddings for each segment.
7. Cosine Similarity is calculated to determine document similarity scores.
8. The Risk Assessment Engine evaluates the scores to generate a detailed report.
9. The results and generated report are displayed back to the teacher via the Flutter interface.

AI-Powered Plagiarism Detection System

System Architecture



Technology Stack

Frontend

- Flutter

Backend

- Python
- FastAPI

AI / NLP

- Sentence-BERT (all-MiniLM-L6-v2)
- Cosine Similarity
- Sentence Transformers
- NLTK / spaCy

Document Processing

- PyMuPDF
- python-docx

Utilities

- Pandas
- NumPy

Detailed Workflow

This section describes the step-by-step user journey and internal processing logic for analyzing document pairs.

- Teacher uploads Document A and Document B.
- Backend extracts text from both documents.
- Text cleaning is performed to remove PDF formatting artifacts and normalize content.
- Academic text is segmented into sentences using NLTK Punkt Tokenizer or spaCy Sentence Boundary Detection.
- SBERT converts each sentence into semantic embeddings.
- Cosine similarity is calculated between sentence embeddings.
- Sentence similarities are aggregated to compute overall document similarity.
- Risk level is assigned based on similarity thresholds.
- Final plagiarism report is generated and displayed.

Report Example

Overall Similarity: 87%

Risk Level: High

Most Similar Sections:

- Sentence 14 ↔ Sentence 22 : 94%
- Sentence 31 ↔ Sentence 18 : 91%
- Sentence 45 ↔ Sentence 52 : 89%

Recommendation:

This document contains highly similar content. Manual review of highlighted sections is recommended before making a final plagiarism determination.